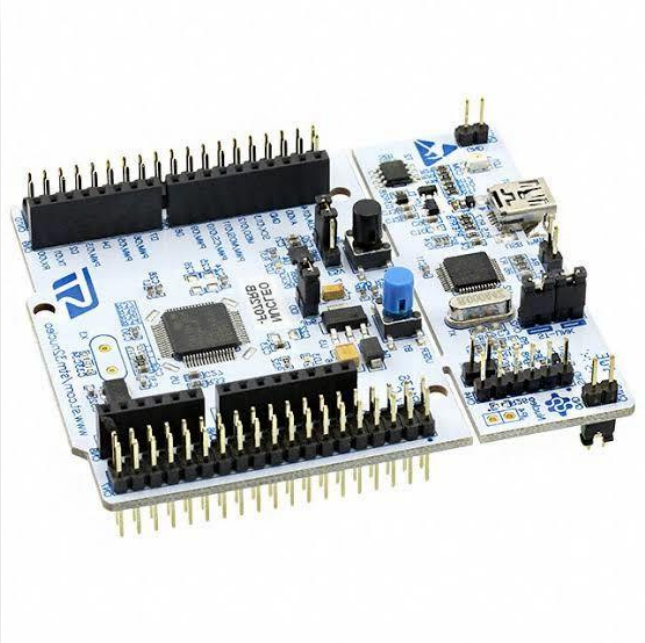
SİVAS CUMHURİYET ÜNİVERSİTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

GÖMÜLÜ SİSTEMLER DERSİ

Gömülü Sistemler Deney 2-B: Analog-Dijital Dönüştürücü (ADC) Uygulamaları (Alternatif Kurulum)

STM32 Nucleo-F072RB, Potansiyometre ve LCD Ekran ile Voltaj Okuma



Dersin Hocası: Doç. Dr. Ahmet Gürkan YÜKSEK
Dersin Asistanı: Arş. Gör. Ahmet Utku ELİK
Öğrenci Adı Soyadı – Numarası:

DENEYİN AMACI VE DEVRE KURULUMU

1. Deneyin Amacı

Bu deneyde, STM32 Nucleo-F072RB geliştirme kartı kullanılarak analog bir sinyalin (Potansiyometre) mikrodenetleyici tarafından okunması (ADC), dijital veriye dönüştürülmesi ve **I2C protokolü** kullanan 16x2 LCD ekran üzerinde görselleştirilmesi amaçlanmaktadır. Bu çalışma ile endüstriyel sensör okuma ve veri gösterme (HMI) uygulamalarının temeli atılmış olacaktır.

2. Kullanılacak Malzemeler

- 1x STM32 Nucleo-F072RB Geliştirme Kartı
- 1x Breadboard (Devre Tahtası)
- 1x 16x02 Karakter LCD Ekran (**I2C Modüllü**)
- 1x 10kΩ Potansiyometre (Analog Sensör Olarak)
- Jumper Kablolar (Erkek-Erkek ve Dişi-Erkek)

3. Devre Bağlantı Şeması

Sistemde I2C haberleşme protokolü kullanıldığı için LCD ekran bağlantısı 4 adet kablo ile sağlanmıştır.

A) I2C LCD Ekran Bağlantısı (Port B)

LCD ekranın arkasındaki modülün pinleri Nucleo kartına aşağıdaki gibi bağlanmalıdır:

I2C Modül Pini	Açıklama	Bağlanacağı Yer (Nucleo / Breadboard)
GND	Toprak	Breadboard **GND (-)** Hattı
VCC	Güç (+5V)	Breadboard **+5V (+)** Hattı
SDA	Veri Hattı	Nucleo **PB7** (veya karttaki SDA pini)
SCL	Saat Hattı	Nucleo **PB6** (veya karttaki SCL pini)

B) Potansiyometre (ADC) Bağlantısı (Port A)

Voltaj değerini değiştirmek için kullanılacak potansiyometre bağlantısı:

Potansiyometre Bacağı	Bağlanacağı Yer
Sol Bacak	Nucleo **+3.3V**
Orta Bacak	Nucleo **PA0** (ADC Girişi)
Sağ Bacak	Breadboard **GND**

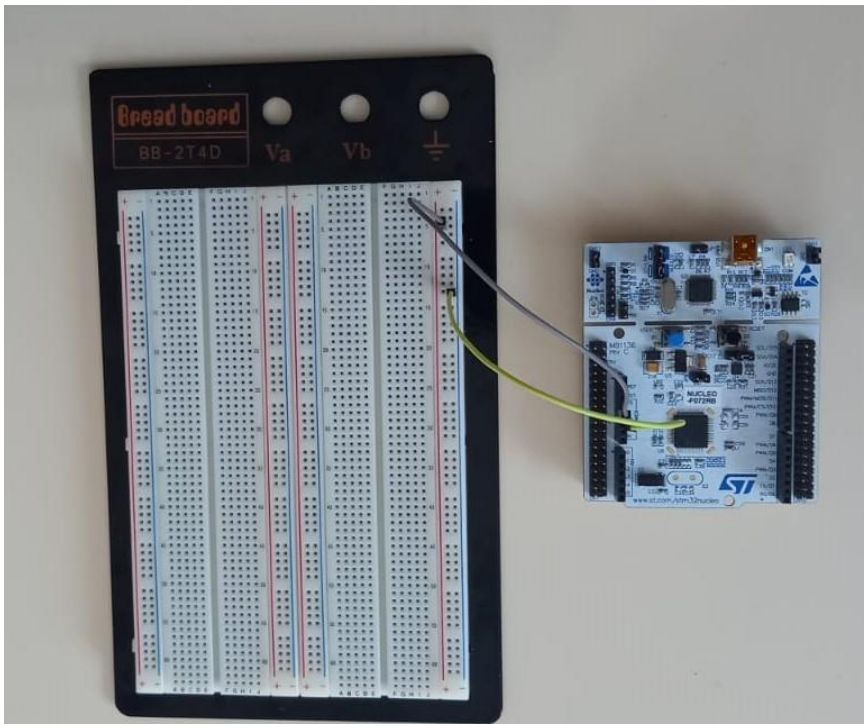
4. Pratik Devre Kurulumu ve Donanım Entegrasyonu

Bu bölümde, teorik şeması verilen devrenin breadboard üzerine fiziksel kurulumu gerçekleştirilecektir. Sinyal gürültüsünü en aza indirmek ve donanım güvenliğini sağlamak için bağlantılar aşağıdaki sıraya göre yapılmalıdır.

Adım 4.1: Güç Dağıtım Hattının (Power Bus) Hazırlanması

Devrenin kararlı çalışması için ortak toprak (Common Ground) ve gerekli gerilim seviyelerinin breadboard üzerine taşınması gerekir.

- **İşlem:** Nucleo kartı üzerindeki **+5V** ve **GND** pinlerinden jumper kablolar çıkartın.
- **Bağlantı:**
  - **GND** kablosunu Breadboard'un **Mavi (-)** hattına takın.
  - **5V** kablosunu Breadboard'un **Kırmızı (+)** hattına takın.
- **Teknik Gerekçe:** LCD ekranın arka aydınlatması ve lojik devresi 5V gerilim seviyesine ihtiyaç duyar. Ayrıca tüm bileşenlerin (MCU, LCD, Sensör) sinyal referansının aynı olması için "Ortak Toprak" şarttır.



Şekil 1: Nucleo kartından breadboard güç raylarına (5V ve GND) enerji aktarımı.

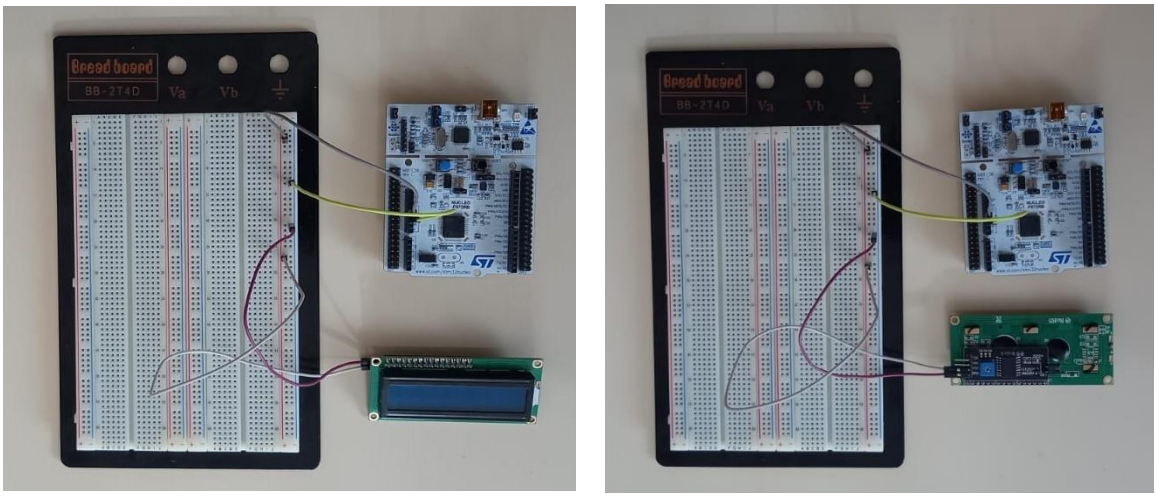
Adım 4.2: I2C LCD Ekranın Haberleşme Bağlantısı

Sistemin çıktı birimi olan LCD ekran, I2C seri haberleşme protokolü ile mikrodenetleyiciye entegre edilecektir.

- **İşlem:** LCD ekranın arkasındaki I2C modülünden çıkan 4 adet kabloyu hazırlayın.

Şekil 2: *Kullanılacak olan 16x2 Karakter LCD ekran ve arkasına entegre edilmiş I2C haberleşme modülü.*

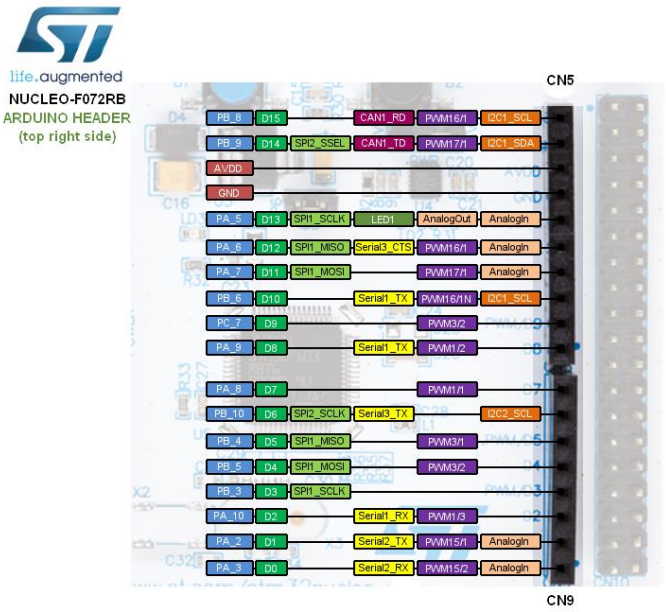
- **Bağlantı:**
  - **VCC & GND:** Breadboard üzerindeki ilgili güç raylarına (+5V ve -) bağlayın.

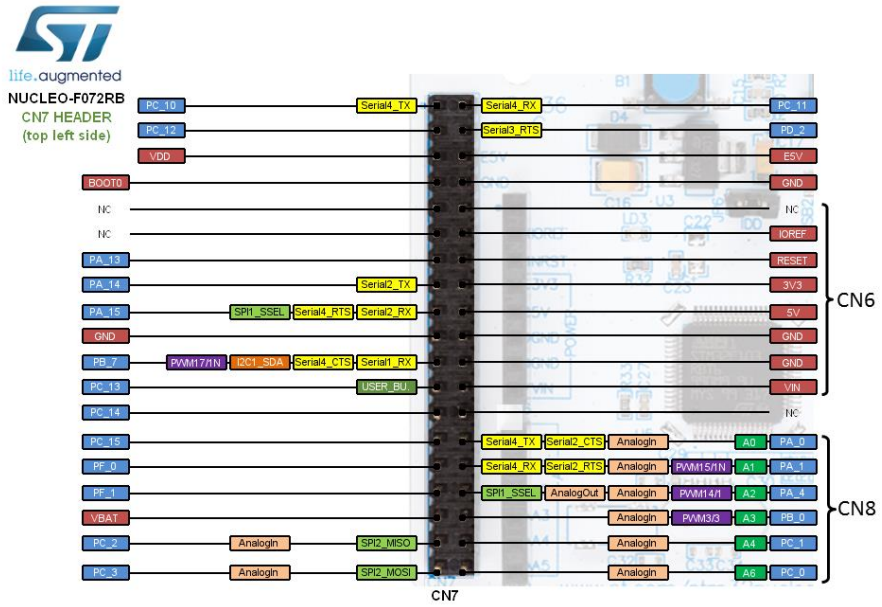


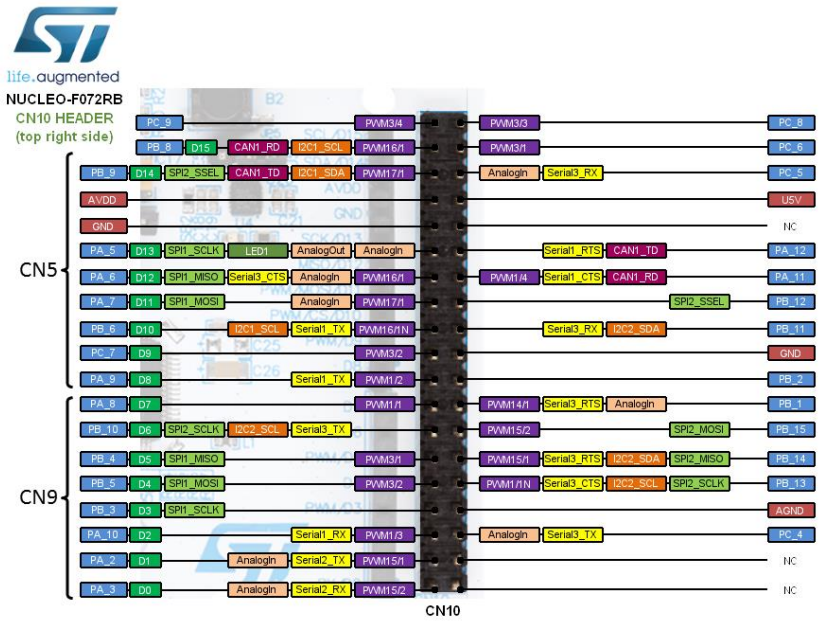
Şekil 3: *I2C LCD modülünün VCC (+5V) ve GND (Toprak) güç bağlantılarının breadboard üzerinde yapılması.*

Fiziksel Pinlerin Tespiti ve Pin Haritası (Pinout)

İşlemcinin LCD ekran ile haberleşeceği (I2C protokolü) ve sensör verisini okuyacağı (ADC) sinyal yollarını bağlamadan önce, Nucleo kartı üzerindeki fiziksel pinlerin yerlerini tespit etmeliyiz. Bunun için **STM32 Nucleo-F072RB Pin Haritası (Pinout)** referans alınır.



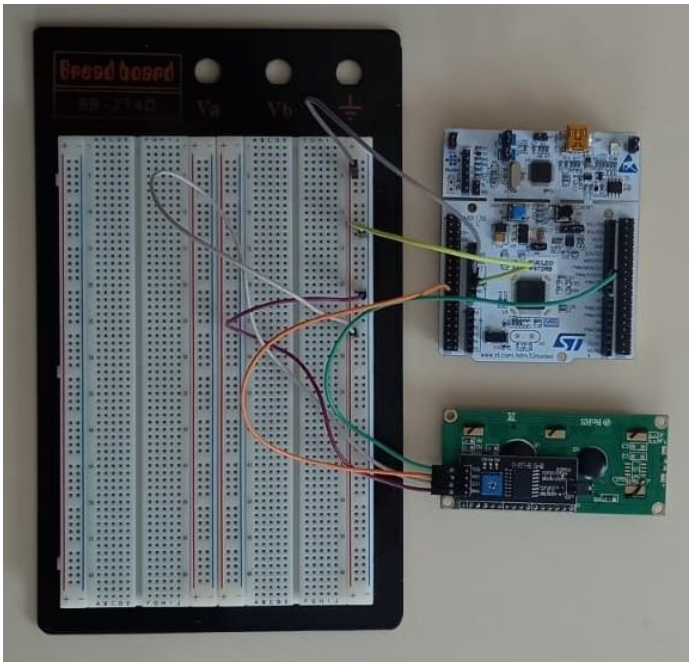




Şekil 4: STM32 Nucleo-F072RB geliştirme kartının Pin Yerleşim Şeması (Pinout) ve Arduino/Morpho konnektör yapısı.

- **SDA (Serial Data):** Veri akışı için Nucleo kartının **PB7** pinine bağlayın.
- **SCL (Serial Clock):** Saat senkronizasyonu için Nucleo kartının **PB6** pinine bağlayın.

- **Teknik Gerekçe:** Standart 16 pinli bağlantı yerine I2C protokolü kullanılarak kablo karmaşası önlenmiş ve Port B üzerindeki pin tasarrufu sağlanmıştır.

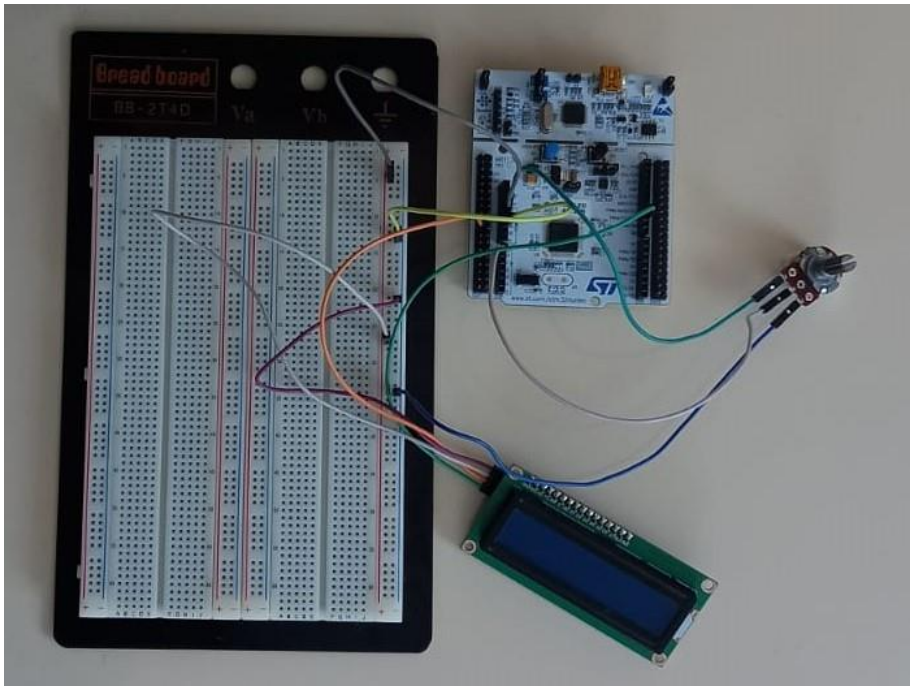


Şekil 5: *LCD haberleşmesi için SDA (PB7) ve SCL (PB6) hatlarının Nucleo kartı ile bağlantısı.*

Adım 4.3: Analog Sensör (Potansiyometre) Arayüzü

Sisteme değişken voltaj girişi sağlamak amacıyla potansiyometre bir "Gerilim Bölücü" (Voltage Divider) olarak devreye alınacaktır.

- **İşlem:** 10kΩ değerindeki potansiyometreyi breadboard üzerine yerleştirin.
- **Bağlantı:**
  - **Sol Bacak:** Nucleo kartı üzerindeki **3.3V** pinine (Dikkat: 5V hattına değil, ADC referans voltajına uygun olmalıdır).
  - **Sağ Bacak:** Breadboard üzerindeki **GND (-)** hattına.
  - **Orta Bacak (Silecek):** Değişken sinyali taşımak için Nucleo **PA0** pinine bağlayın.
- **Teknik Gerekçe:** Potansiyometre, mekanik hareketi direnç değişimine çevirerek orta bacakta 0V ile 3.3V arasında lineer bir analog gerilim oluşturur. Bu gerilim, MCU'nun ADC birimi tarafından işlenecektir.

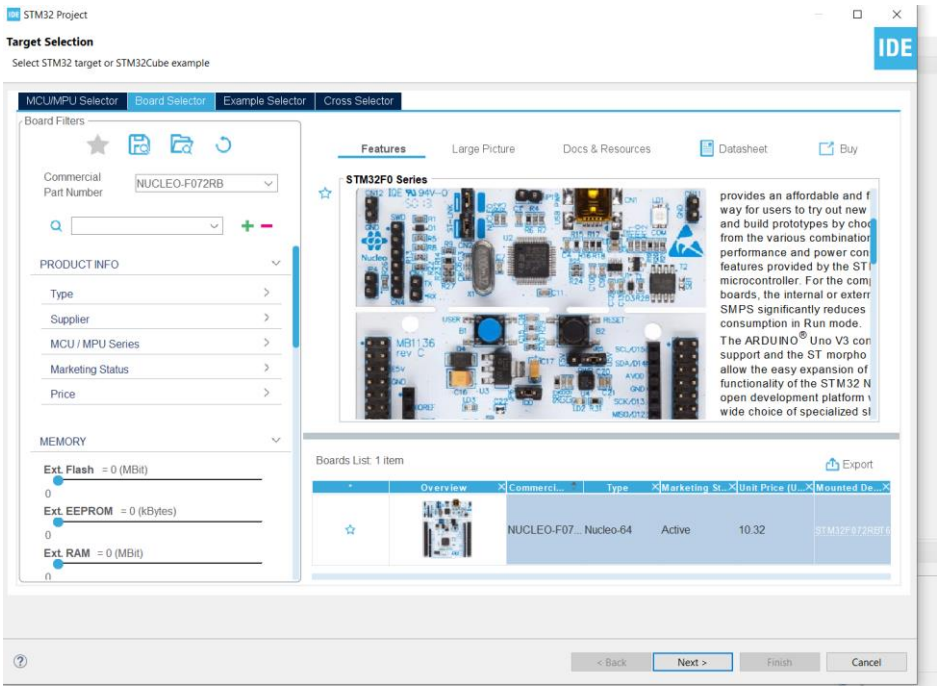


Şekil 6: *Analog veri okuması için potansiyometrenin PA0 pinine bağlanması ve gerilim bölücü devresi.*

5. STM32CubeIDE KONFİGÜRASYONU (Nucleo İçin)

Adım 1: Proje Oluşturma

1. STM32CubeIDE'yi açın. File > New > STM32 Project seçeneğine tıklayın.
2. **Board Selector** sekmesine gelin. Arama kutusuna **"NUCLEO-F072RB"** yazın.
3. Sağdaki listeden kartı seçip **Next** deyin. Proje ismine **" Deney2B_Nucleo_ADC_I2C"** yazın.

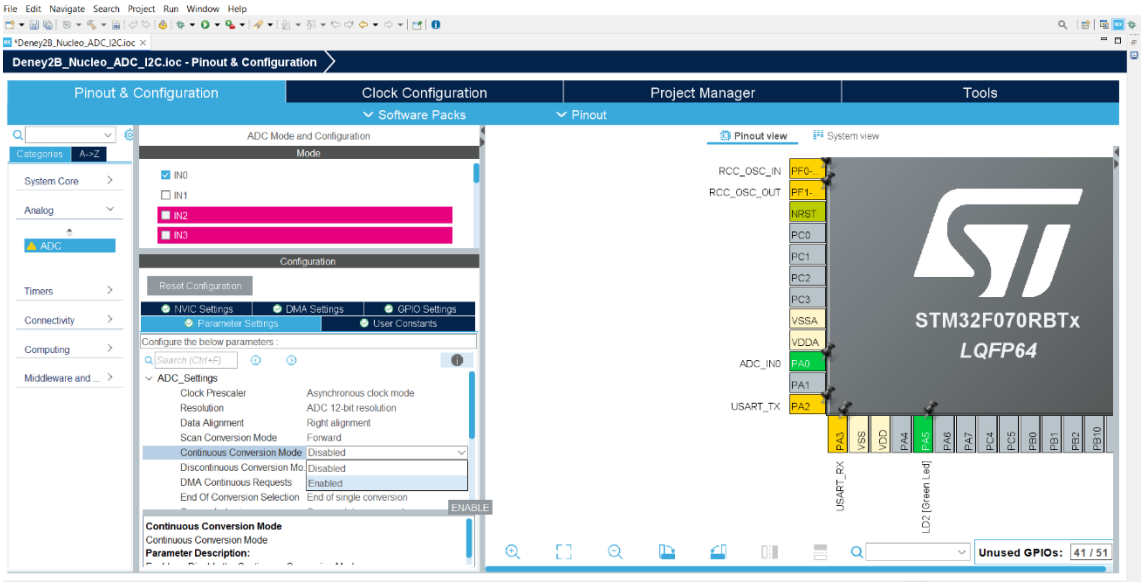


Şekil 7: *STM32CubeIDE "Board Selector" ekranında NUCLEO-F072RB kartının seçilmesi.*

4. **"Initialize all peripherals with their default Mode?"** sorusu gelince **"No"** deyin.

Adım 2: ADC (Analog Giriş) Ayarları Potansiyometreyi okumak için ADC birimini açmalıyız.

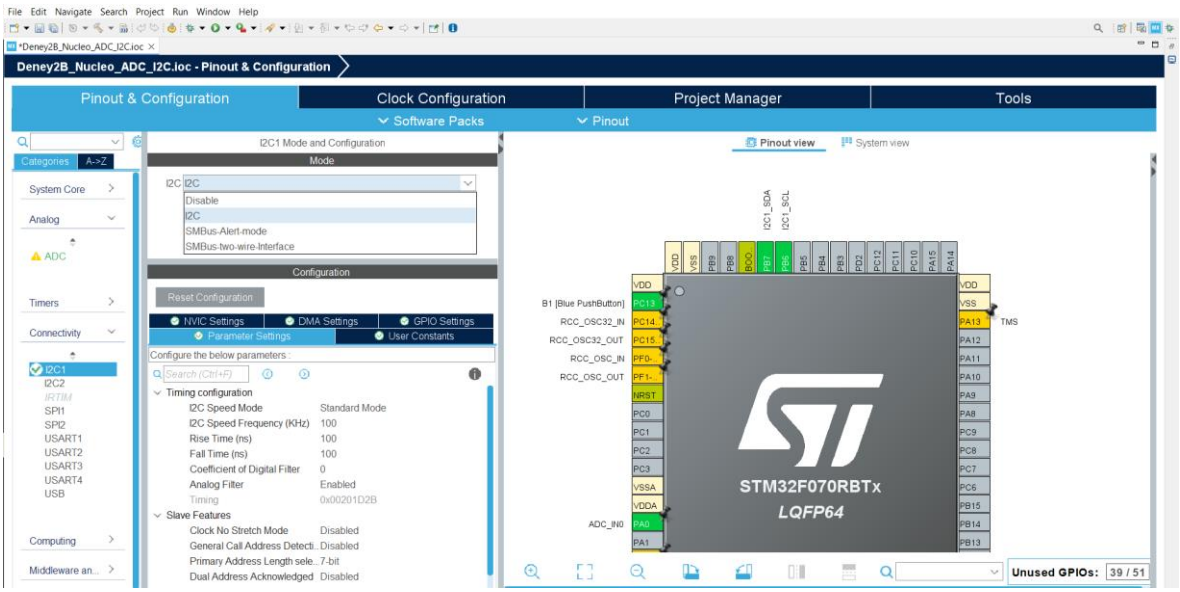
1. Pinout ekranında **PA0** pinine sol tıklayın ve **ADC_IN0** seçeneğini işaretleyin.
2. Sol menüden **Analog > ADC** sekmesine gelin.
3. **IN0** kutucuğunu (check box) işaretleyin.
4. Aşağıdaki **Configuration** penceresinde şu ayarları kontrol edin:
   - **Scan Conversion Mode:** *Forward* (veya açık kalsın)
   - **Continuous Conversion Mode: Enabled** (Bu çok önemli! Sürekli okuma yapacağız).



Şekil 8: ADC biriminin aktif edilmesi (IN0) ve "Continuous Conversion Mode" (Sürekli Okuma) ayarının yapılması.

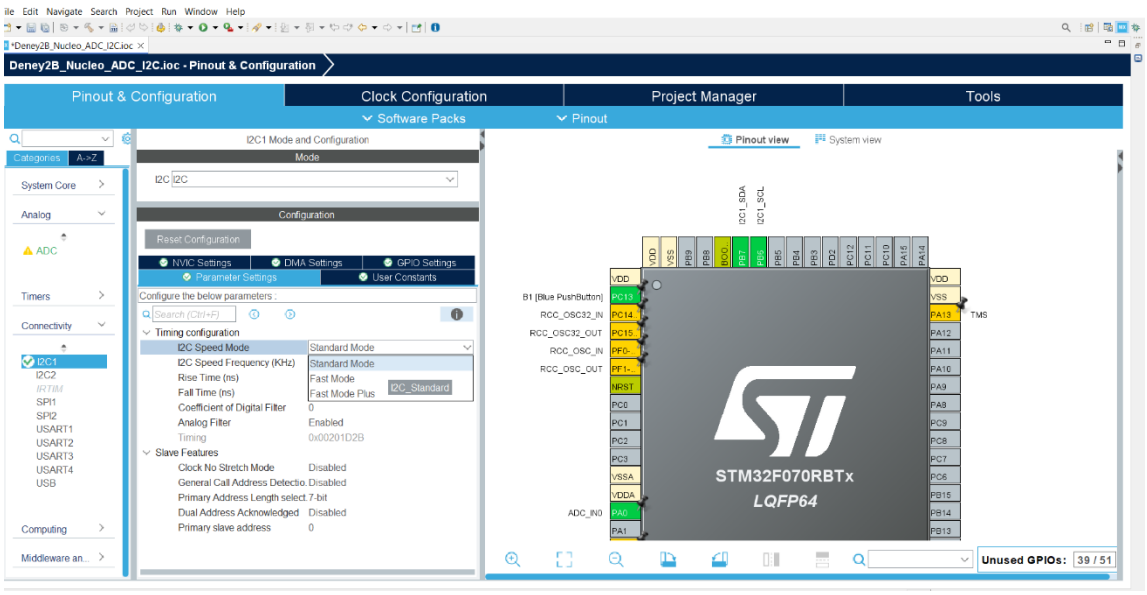
Adım 3: I2C (LCD Haberleşme) Ayarları LCD ekranla konuşmak için I2C protokolünü açmalıyız.

1. Sol menüden **Connectivity** > **I2C1** sekmesine gelin.
2. **I2C1** (veya Disable yazan yer) açılır menüsünden **I2C** seçeneğini seçin.
3. Sağ taraftaki çip üzerinde **PB6 (SCL)** ve **PB7 (SDA)** pinlerinin otomatik yeşil olduğunu görün. (Eğer başka pinler yeşil olduysa, Ctrl+Click ile PB6/PB7'ye taşıyabilirsin ama genelde otomatik burayı seçer).



Şekil 9: Bağlantı (Connectivity) sekmesinden I2C1 arayüzünün aktif edilmesi ve PB6/PB7 pin atamaları.

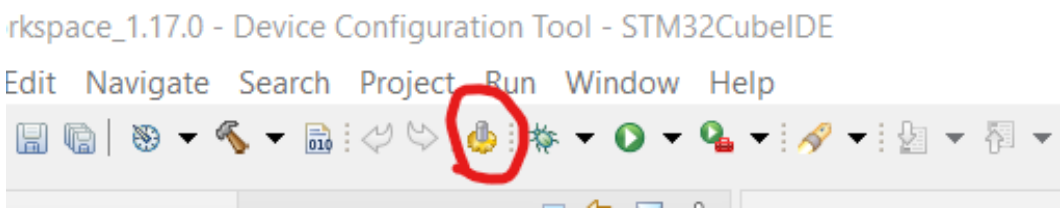
4. **Configuration > Parameter Settings** kısmında:
   - **I2C Speed Mode:** *Standard Mode*
   - **Speed Frequency:** *100 kHz* (LCD'ler hızlı konuşmayı sevmez, standart hız iyidir).



Şekil 10: I2C parametre ayarlarında haberleşme hızının "Standard Mode" (100 kHz) olarak yapılandırılması.

Adım 4: Kod Üretimi

- Üst menüdeki **Sarı Dişli (Device Configuration Tool Code Generation)** ikonuna basarak `main.c` dosyasını oluşturun.



Şekil 11: Yapılan konfigürasyonun C koduna dönüştürülmesi için "Code Generation" (Sarı Dişli) işleminin başlatılması.

6. Yazılım Geliştirme ve Kütüphane Entegrasyonu

Harici LCD Sürücü Kütüphanesinin Entegrasyonu

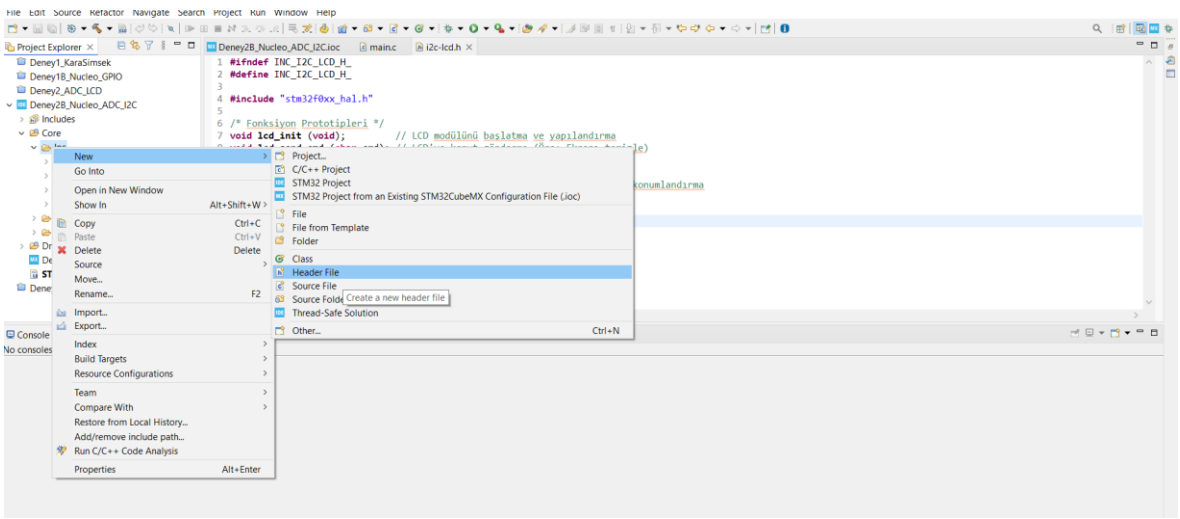
STM32 HAL (Hardware Abstraction Layer) kütüphanesi, standart karakter LCD modülleri için yerleşik sürücüler içermemektedir. Bu nedenle, I2C protokolü üzerinden LCD kontrolünü sağlayan harici sürücü dosyalarının (*Driver Layer*) projeye manuel olarak dahil edilmesi gerekmektedir.

Aşağıdaki adımlar takip edilerek `i2c-lcd.h` (Başlık) ve `i2c-lcd.c` (Kaynak) dosyaları projeye eklenmelidir.

Adım 6.1: Başlık Dosyasının (Header File) Oluşturulması

Bu dosya, LCD fonksiyonlarının prototiplerini içerir ve diğer kod dosyalarının bu fonksiyonlara erişmesini sağlar.

1. STM32CubeIDE proje ağacında **`Core > Inc`** klasörüne sağ tıklanır.
2. **New** > **Header File** seçeneği seçilir.



Şekil 12: *Harici kütüphane entegrasyonu: "Core > Inc" dizini altında "i2c-lcd.h" başlık dosyasının oluşturulması.*

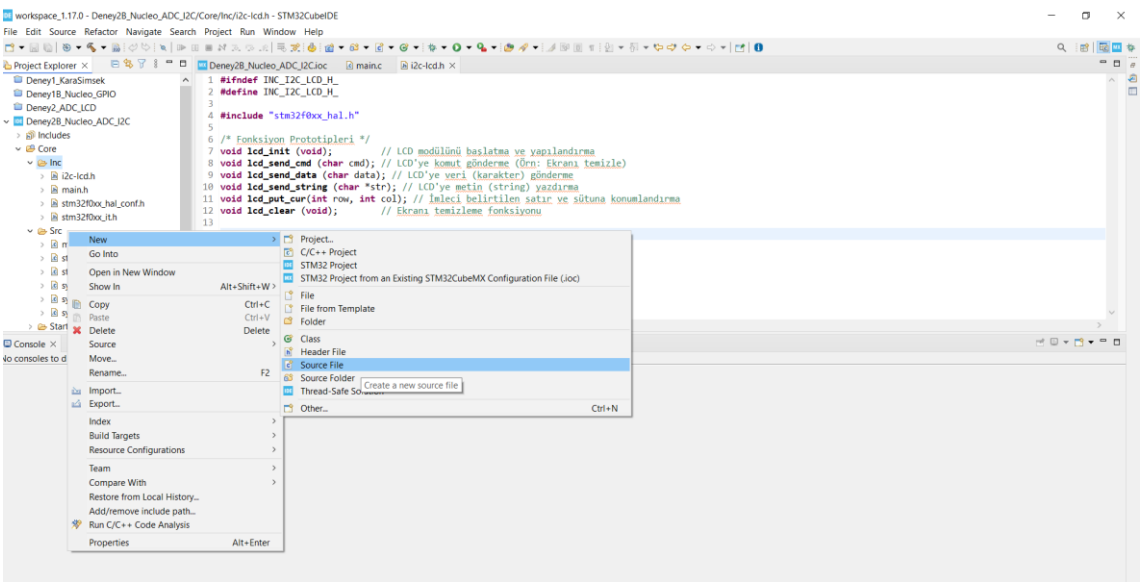
3. Dosya adı olarak **`i2c-lcd.h`** girilir ve **Finish** butonuna basılır.
4. Oluşturulan dosyanın içeriği tamamen silinerek aşağıdaki kod bloğu eklenir:

```
#ifndef INC_I2C_LCD_H_
#define INC_I2C_LCD_H_

#include "stm32f0xx_hal.h"

/* Fonksiyon Prototipleri */
void lcd_init (void);          // LCD modülünü başlatma ve yapılandırma
void lcd_send_cmd (char cmd); // LCD'ye komut gönderme (Örn: Ekranı temizle)
void lcd_send_data (char data); // LCD'ye veri (karakter) gönderme
void lcd_send_string (char *str); // LCD'ye metin (string) yazdırma
void lcd_put_cur(int row, int col); // İmleci belirtilen satır ve sütuna
konumlandırma
void lcd_clear (void);         // Ekranı temizleme fonksiyonu

#endif /* INC_I2C_LCD_H_ */
```

Adım 6.2: Kaynak Dosyasının (Source File) Oluşturulması

Bu dosya, başlık dosyasında tanımlanan fonksiyonların gövdesini (işlevlerini) ve I2C haberleşme algoritmalarını içerir.

1. Proje ağacında `Core > Src` klasörüne sağ tıklanır.
2. **New** > **Source File** seçeneği seçilir.



Şekil 13: Harici kütüphane entegrasyonu: "Core > Src" dizini altında "i2c-lcd.c" kaynak dosyasının oluşturulması.

3. Dosya adı olarak `i2c-lcd.c` girilir ve **Finish** butonuna basılır.
4. Dosya içeriği temizlenerek aşağıdaki kod bloğu eklenir

```c
#include "i2c-lcd.h"

/* main.c dosyasında tanımlanan I2C birimine erişim */
extern I2C_HandleTypeDef hi2c1;

/* I2C Modül Adresi (Slave Address) */
/* NOT: Kullanılan modüle göre bu adres 0x27 veya 0x4E olabilir */
#define SLAVE_ADDRESS_LCD 0x4E

/* LCD'ye Komut Gönderme Fonksiyonu */
void lcd_send_cmd (char cmd)
{
  char data_u, data_l;
  uint8_t data_t[4];
  data_u = (cmd&0xf0);
  data_l = ((cmd<<4)&0xf0);
  data_t[0] = data_u|0x0C;  //en=1, rs=0
  data_t[1] = data_u|0x08;  //en=0, rs=0
  data_t[2] = data_l|0x0C;  //en=1, rs=0
  data_t[3] = data_l|0x08;  //en=0, rs=0
  HAL_I2C_Master_Transmit (&hi2c1, SLAVE_ADDRESS_LCD,(uint8_t *) data_t, 4, 100);
}

/* LCD'ye Veri Gönderme Fonksiyonu */
void lcd_send_data (char data)
{
  char data_u, data_l;
  uint8_t data_t[4];
  data_u = (data&0xf0);
  data_l = ((data<<4)&0xf0);
  data_t[0] = data_u|0x0D;  //en=1, rs=1
  data_t[1] = data_u|0x09;  //en=0, rs=1
  data_t[2] = data_l|0x0D;  //en=1, rs=1
  data_t[3] = data_l|0x09;  //en=0, rs=1
  HAL_I2C_Master_Transmit (&hi2c1, SLAVE_ADDRESS_LCD,(uint8_t *) data_t, 4, 100);
}

/* Ekran Temizleme Fonksiyonu */
void lcd_clear (void)
{
      lcd_send_cmd (0x80);
      for (int i=0; i<70; i++)
      {
            lcd_send_data (' ');
      }
}
```

```
/* İmleç Konumlandırma Fonksiyonu */
void lcd_put_cur(int row, int col)
{
    switch (row)
    {
        case 0:
            col |= 0x80;
            break;
        case 1:
            col |= 0xC0;
            break;
    }
    lcd_send_cmd (col);
}

/* LCD Başlatma (Initialization) Algoritması */
void lcd_init (void)
{
      // 4-bit Modu Başlatma Sırası
      HAL_Delay(50);
      lcd_send_cmd (0x30);
      HAL_Delay(5);
      lcd_send_cmd (0x30);
      HAL_Delay(1);
      lcd_send_cmd (0x30);
      HAL_Delay(10);
      lcd_send_cmd (0x20);
      HAL_Delay(10);

      // Ekran Konfigürasyonu
      lcd_send_cmd (0x28); // 4-bit modu, 2 satır, 5x8 font
      HAL_Delay(1);
      lcd_send_cmd (0x08); // Ekran Kapalı
      HAL_Delay(1);
      lcd_send_cmd (0x01); // Ekranı Temizle
      HAL_Delay(1);
      HAL_Delay(1);
      lcd_send_cmd (0x06); // İmleç artış modu
      HAL_Delay(1);
      lcd_send_cmd (0x0C); // Ekran Açık, İmleç Kapalı
}

/* String Yazdırma Fonksiyonu */
void lcd_send_string (char *str)
{
      while (*str) lcd_send_data (*str++);
}
```

7. main.c KODLAMASI

1. Kütüphaneleri Dahil Etme

Dosyanın en üst kısımlarına çık. `/* USER CODE BEGIN Includes */` yazan yeri bul ve arasına şunları ekle:

```
/* USER CODE BEGIN Includes */
#include "i2c-lcd.h"
#include <stdio.h>  // sprintf fonksiyonu ile metin birleştirmek için gerekli
/* USER CODE END Includes */
```

2. Değişkenleri Tanımlama

`main` fonksiyonuna gelmeden hemen önceki `/* USER CODE BEGIN PV */` (Private Variables) kısmını bul ve değişkenlerimizi tanımla:

```
/* USER CODE BEGIN PV */
uint32_t adc_val = 0;   // ADC'den okunan ham değer (0 - 4095 arası)
float voltage = 0.0;    // Hesaplanacak voltaj değeri (0.00V - 3.30V arası)
char lcd_buffer[20];    // LCD'ye yazılacak metni tutan hafıza
/* USER CODE END PV */
```

3. Başlangıç Ayarları ve Sonsuz Döngü

Şimdi `int main(void)` fonksiyonunun içine gir. `/* USER CODE BEGIN 2 */` kısmını bul (while döngüsünden hemen önce) ve başlatma kodlarını yaz:

```
/* USER CODE BEGIN 2 */
  lcd_init();                     // LCD ekranı başlat
  lcd_send_string("Sivas CU");    // Açılış mesajı 1. satır
  lcd_put_cur(1, 0);              // Alt satıra geç
  lcd_send_string("Müh. Fakultesi");// Açılış mesajı 2. satır
  HAL_Delay(2000);                // 2 saniye bekle
  lcd_clear();                    // Ekranı temizle

  HAL_ADC_Start(&hadc);           // ADC birimini başlat
  /* USER CODE END 2 */
```

Hemen altındaki `while (1)` döngüsünün içine, yani `/* USER CODE BEGIN 3 */` kısmına ana mantığı yapıştır:

```
/* USER CODE BEGIN 3 */
        // 1. ADC'den Veri Okuma
        HAL_ADC_PollForConversion(&hadc, 100); // Dönüşümün bitmesini bekle
        adc_val = HAL_ADC_GetValue(&hadc);     // Ham değeri (0-4095) oku

        // 2. Voltaj Dönüşümü (Matematiksel Hesaplama)
        // Formül: (Okunan Değer * Referans Voltajı) / Çözünürlük
        voltage = (adc_val * 3.3) / 4095.0;

        // 3. LCD'ye Yazdırma

        // 1. Satır: Ham ADC Değeri
        lcd_put_cur(0, 0);
        sprintf(lcd_buffer, "Raw ADC: %lu   ", adc_val); // "Raw ADC: 2048" gibi
yazar
        lcd_send_string(lcd_buffer);

        // 2. Satır: Hesaplanan Voltaj
        lcd_put_cur(1, 0);
        sprintf(lcd_buffer, "Volt: %.2f V   ", voltage); // "Volt: 1.65 V" gibi
yazar
        lcd_send_string(lcd_buffer);

        HAL_Delay(200); // Gözün takip edebilmesi için 200ms bekle
  }
  /* USER CODE END 3 */
```

Gerekli kütüphaneler projeye eklendikten sonra `main.c` dosyasında ana kontrol algoritması oluşturulmuştur. Yazılımın akış diyagramı şu şekildedir:

1. **Başlatma (Initialization):** Sistem açıldığında `lcd_init()` fonksiyonu ile LCD ekran kullanıma hazır hale getirilir ve kullanıcıya bir açılış mesajı gösterilir. `HAL_ADC_Start()` fonksiyonu ile ADC birimi tetiklenir.
2. **Veri Okuma (Polling):** Sonsuz döngü içerisinde `HAL_ADC_PollForConversion` komutu ile ADC çevriminin tamamlanması beklenir ve `HAL_ADC_GetValue` ile 12-bitlik ham dijital veri (0-4095 arası) okunur.
3. Veri İşleme: Okunan ham veri, aşağıdaki formül kullanılarak anlamlı bir gerilim değerine dönüştürülür:

$$Vout = \frac{ADC * 3.3V}{4095}$$

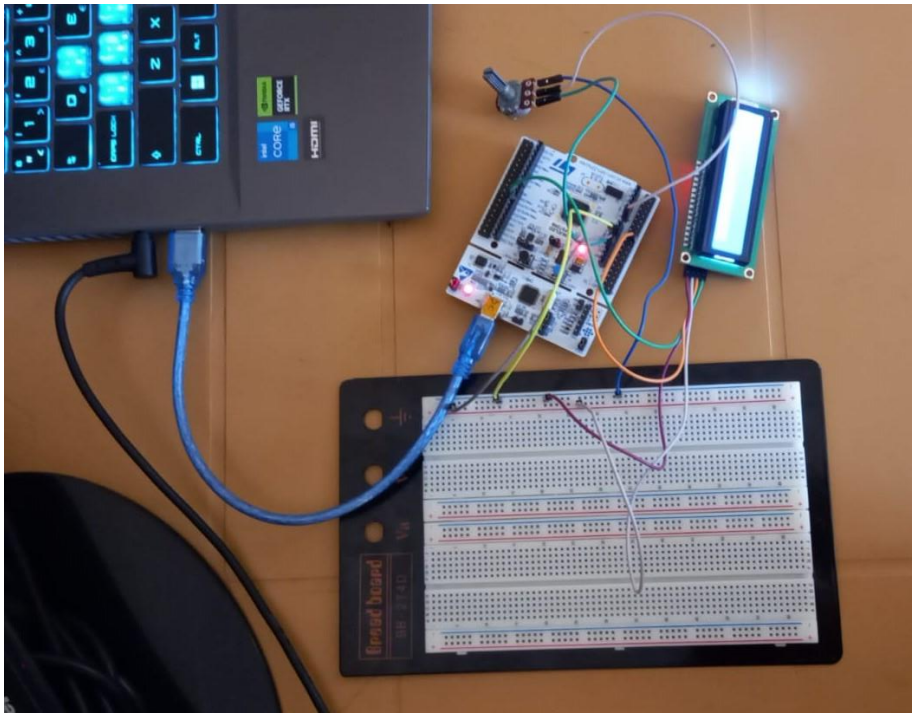
4. **Görselleştirme:** Elde edilen ham veri ve hesaplanan voltaj değeri, `sprintf` fonksiyonu ile metin formatına çevrilerek I2C LCD ekranın 1. ve 2. satırlarına yazdırılır.

Pratik Uygulama ve Test Adımları

Yazdığımız kodun STM32 Nucleo kartına yüklenmesi ve breadboard üzerindeki devrenin test edilmesi için aşağıdaki adımları izleyin.

Adım 1: Fiziksel Bağlantı ve Kontrol

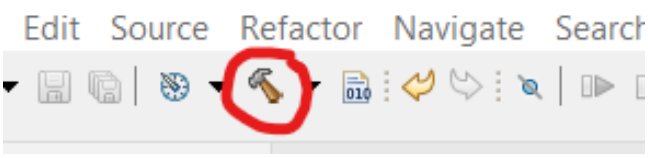
1. Kurduğunuz breadboard devresindeki kablo bağlantılarını son kez kontrol edin. Özellikle **GND** hattının Nucleo ile birleştiğinden emin olun.
2. STM32 Nucleo-F072RB geliştirme kartının üst kısmında bulunan USB (ST-LINK) girişine kabloyu takın.
3. Kablonun diğer ucunu bilgisayarınızın USB portuna bağlayın.
4. Kart üzerindeki **PWR (Güç)** LED'inin (genellikle Yeşil) yandığını ve yanındaki **LD1 (COM)** LED'inin yanıp söndüğünü doğrulayın.



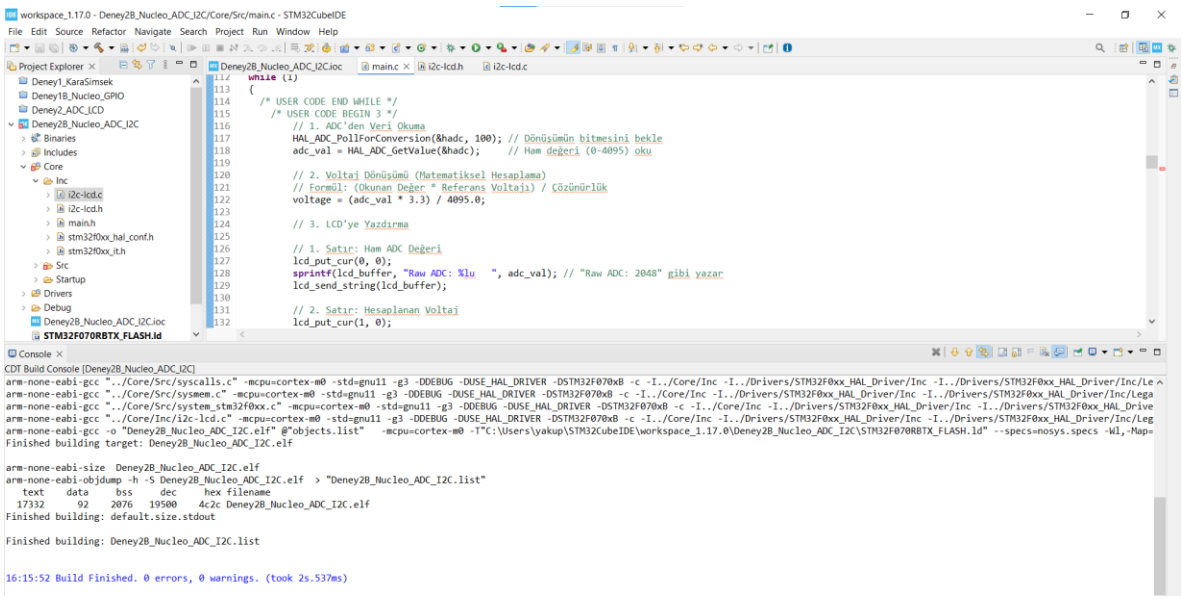
Şekil 14: Yazılımın yüklenmesi için Nucleo kartının USB kablosu ile bilgisayara fiziksel bağlantısı.

Adım 2: Kodu Derleme ve Yükleme

1. STM32CubeIDE üst menüsünde bulunan **Çekiç (Build)** ikonuna tıklayın ve "0 Errors" mesajını görene kadar bekleyin.

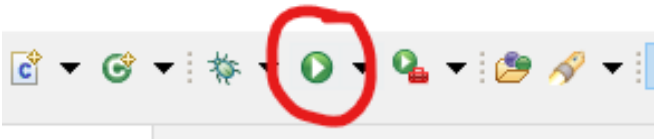


Şekil 15: Yazılan kodun makine diline çevrilmesi için Derleme (Build) işleminin gerçekleştirilmesi.



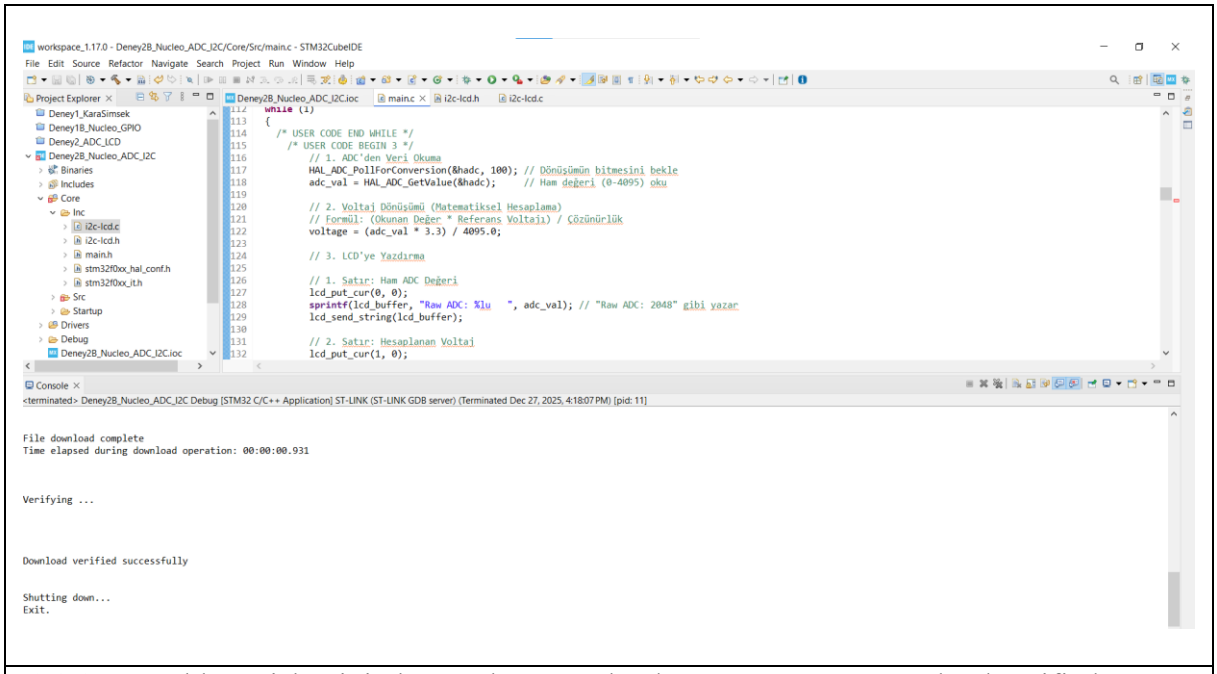
Şekil 16: Derleme işleminin hatasız tamamlandığını doğrulayan "0 Errors, 0 Warnings" konsol çıktısı.

2. **Yeşil Oynat (Run)** butonuna tıklayın.



Şekil 17: Derlenen yazılımın mikrodenetleyiciye yüklenmesi için "Run" butonunun kullanılması.

3. Açılan pencerede kartın seçili olduğunu görüp **OK** butonuna basın. Yükleme tamamlandığında konsolda "Download verified successfully" yazısını görmelisiniz.



Şekil 18: Yükleme işleminin başarıyla tamamlandığını gösteren "Download verified successfully" konsol mesajı.

8. FONKSİYON TESTLERİ VE GÖZLEM

Yazılımın karta yüklenmesinin ardından sistemin kararlılığını ve kullanıcı arayüzü tepkilerini doğrulamak için aşağıdaki test senaryoları sırasıyla uygulanmıştır:

Test 1: Sistem Başlatma ve Açılış Ekranı Testi

Nucleo kartına enerji verildiğinde veya Siyah (Reset) butona basıldığında sistemin davranışıdır.

- **Beklenen:** LCD ekranın başlatılması (Initialization) ve kodda belirlenen açılış mesajının görünmesi.
- **Gözlem:** Enerji verildikten hemen sonra ekranın 1. satırında **"Sivas CU"**, 2. satırında **"Müh. Fakultesi"** yazısı 2 saniye boyunca görüntülenmiş, ardından ekran temizlenerek voltaj okuma moduna geçilmiştir.

Test 2: Sıfır Noktası Testi (GND)

Potansiyometre en sola (minimum konuma) çevrilmiştir.

- **Beklenen:** ADC girişinin 0V seviyesini algılaması.
- **Gözlem:** LCD ekranda **"Raw: 0"** ham verisi ve **"Volt: 0.00 V"** gerilim değeri stabil olarak okunmuştur.

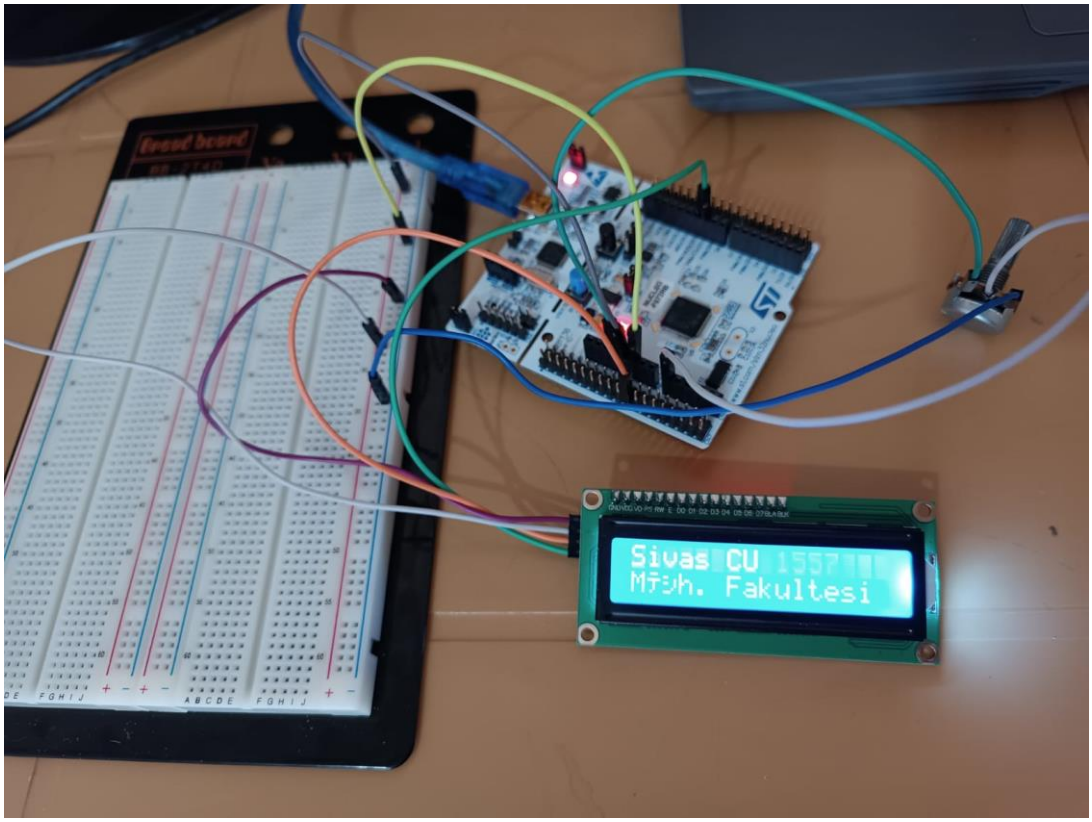
Test 3: Tepe Noktası Testi (3.3V)

Potansiyometre en sağa (maksimum konuma) çevrilmiştir.

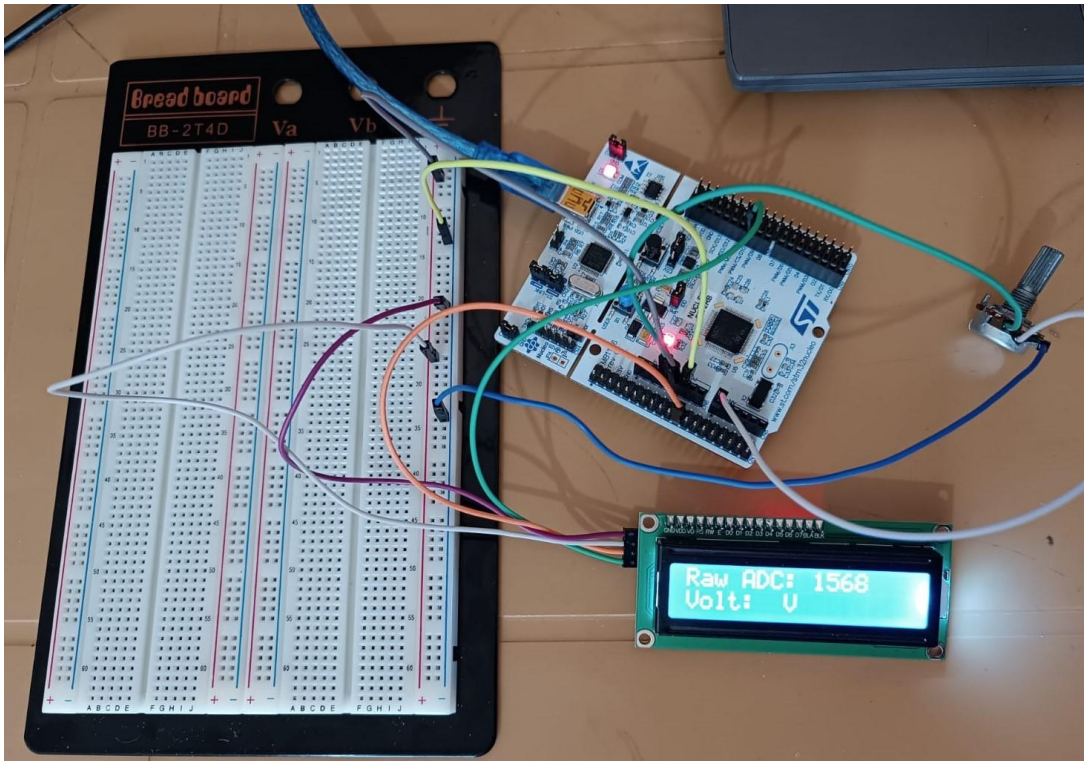
- **Beklenen:** ADC'nin 12-bitlik üst limiti olan 4095 değerine ulaşılması.
- **Gözlem:** LCD ekranda **"Raw: 4095"** (veya 4090-4095 arası) ham veri ve **"Volt: 3.30 V"** gerilim değeri doğrulanmıştır.

Test 4: Hassasiyet ve Tepki Süresi Testi

Potansiyometre yavaşça çevrildiğinde voltaj değişiminin ekrana anlık yansıdığı, I2C haberleşmesinde herhangi bir donma veya gecikme olmadığı test edilmiştir.



Şekil 19: Sistem açılışında (Initialization) kullanıcıyı karşılayan "Sivas CU" ve "Müh. Fakultesi" bilgilendirme ekranı.



Şekil 20: *Sistem testi: Potansiyometre minimum konumdayken (0V) LCD ekranda okunan dijital ve analog değerler.*